ATTENDANCE MANAGEMENT SYSTEM

A Project submitted to the

BHARATHIDASAN UNIVERSITY, THIRUCHIRAPPALLI

In Partial fulfillment of the requirements for the award of degree of

BACHELOR OF COMPUTER SCIENCE

Submitted by

N.Angalaparameshwari (Register No:23050511802112001)

M.Chithkala (Register No.23050511802112003)

S.Dhivya (Register No.23050511802112005)

Under the Guidance of

Mrs.S.KAVITHA, MCA,

Head of the department

UG DEPARTMENT OF COMPUTER SCIENCE

SRI SARADA NIKETAN COLLEGE OF SCIENCE FOR WOMEN

KARUR–5

APRIL 2026

CERTIFICATE

CERTIFICATE

This is to certify that the project work done by my guidance and project entitled "**ATTENDANCE MANAGEMENT SYSTEM**" in partial fulfillment of the requirement for the award of degree of "**BACHELOR OF COMPUTER SCIENCE**" is a record of the original work done by **N.ANGALAPARAMESHWARI Register No.23050511802112001, M.CHITHKALA Register No.23050511802112003, S.DHIVYA Register No.23050511802112005** from 2023–2026 in **DEPARTMENT OF COMPUTER SCIENCE, Sri Sarada Niketan College of Science for Women, Karur–5** under my supervision and guidance.

Signature of the candidate Signature of the Guide

Signature of the Head of the Department Signature of the Principal

Submitted for the VIVA-VOCE Examination held on……

Signature of the Examiner

1.

2.

DECLARATION

DECLARATION

I hereby declare that this project report entitled "**ATTENDANCE MANAGEMENT SYSTEM"** submitted to Bharathidasan University,Tiruchirappalli in partial fulfillment of the requirement for the award of degree of **"BACHLORE OF COMPUTER SCIENCE"** is a record of bonafide project work done by me during the period of study in the Department of Computer Science, Sri Sarada Niketan College for Women under the supervision of **Mrs.S.KAVITHA, MCA.**, **Head and Assistant Professor in Department of Computer Science, Sri Sarada Niketan College of Science for Women, Karur-5.**

N. ANGALAPARAMESHWARI

M. CHITHKALA

S. DHIVYA

ACKNOWLEDGEMENT

ACKNOWLEDEMENT

I thank **GOD** the almighty who are a great inspiration throughout my endeavor to complete this work of mine successfully.

I express and foremost acknowledge my gratitude to **SRIMATHSWAMI SATHYANANDA**, Vice President, Sri Ramakrishna Tapovanam, Thiruparaithurai for his blessings and holy spirit in completing our project.

I am desired with whole hearted gratitude by my holiness **YATEESWARI NEELAKANTA PRIYA AMBA, Secretary, Sri Sarada NiketanCollege of Science for Women, Karur** for extending the facilities needed to this project.

I sincerely thank our Principal **Dr.S.KAVITHA, M.Com.,M.Phil.,PGDCA,Ph.D., The Principal, Sri Sarada Niketan College of Science for Women, Karur** for this encouragement to carry out this project successfully.

I express my thanks to **Mrs.S.KAVITHA, MCA., Head and Assistant Professor, Department of Computer Science,** Sri Sarada Niketan College of Science forWomen, Karur who encouraged me to complete the project in time. For her benevolent guidance in failing encouragement and untiring help critical reading of them and us script to this project in a better way.

I like to extend my gratitude towards the staff members of my department and librarian who provided all the materials regarding this project work.

I specially convey my thanks to **OUR PARENTS AND FRIENDS** who supported me in all circumstances for the successful completion of my project work.

CONTENTS

CONTENTS

S.NO	TITLE	PAGE NO
1	**INTRODUCTION**	14
2	**SYSTEM ANALYSIS**	16
	2.1 Existing System	16
	2.2 Proposed System	16
	2.3 Feasibility Study	16
3	**SYSTEM SPECIFICATION**	19
	3.1 Hardware Requirements	19
	3.2 Software Requirements	19
	3.3 Software Description	19
4	**SYSTEM DESIGN**	22
	4.1 Input Design	22
	4.2 Output Design	23
	4.3 System Architecture	25
5	**SYSTEM IMPLEMENTATION**	27
	5.1 System Development	27
	5.2 Database Implementation	27
	5.3 System Deployment	28
	5.4 User Training and Maintenance	28
6	**SYSTEM TESTING**	30
	6.1 Unit Testing	30
	6.2 System Testing	30

	6.3 User Acceptance Testing	31
7	**CONCLUSION**	33
8	**FUTURE ENHANCEMENT**	35
9	**BIBLIOGRAPHY**	37
10	**APPENDIX**	39
	10.1 Sample Code	39
	10.2 Sample Output	44

ABSTRACT

ABSTRACT

The Attendance Management System is a console-based mini project developed using the Python programming language to automate and simplify the process of recording student attendance in educational institutions. The main objective of this system is to reduce manual effort, minimize errors, and maintain organized attendance records efficiently.

This project is developed using fundamental Python concepts such as conditional statements, loops, dictionaries, functions, and object-oriented programming. A Student class is used to manage student data, while a separate function handles the attendance process. Dictionaries are utilized to store attendance information efficiently.

The system is simple, user-friendly, and suitable for small-scale academic environments. Although it does not include database connectivity or a graphical user interface, it provides a strong foundation for understanding attendance management logic and object-oriented programming concepts. The project can be further enhanced by adding database integration, graphical interfaces, and automated report generation features.

Overall, this Attendance Management System demonstrates the practical application of Python programming in solving real-world academic management tasks efficiently and effectively.

INTRODUCTION

1. INTRODUCTION

The **Attendance Management System** is a Python-based console application developed to simplify the process of recording and managing student attendance. This system allows the user to select academic details such as country, state, district, college, and department. After selecting the required academic structure, student details can be entered and attendance can be marked as Present or Absent.

The system automatically calculates the total number of students present and absent and displays the attendance report clearly. It also demonstrates the use of object-oriented programming by implementing a Student class to manage student information effectively.

This project is developed using basic Python concepts such as conditional statements, loops, dictionaries, functions, and classes. It is designed to be simple, user-friendly, and suitable for small academic institutions or as a beginner-level programming project.

The main purpose of an Attendance Management System is to reduce manual work, minimize errors, and maintain accurate attendance records. It allows administrators or teachers to easily monitor attendance, generate reports, and analyze patterns such as absenteeism or punctuality.

In educational institutions and organizations, an AMS improves transparency, saves time, and ensures better record management. By storing attendance data electronically, the system also makes it easier to retrieve past records and integrate them with other management systems.

(AMS) is a digital solution used to record, track, and manage the attendance of students or employees efficiently. Instead of using traditional paper registers, this system automates the attendance process using technologies such as databases, biometric devices, RFID cards, or web/mobile applications.

SYSTEM ANALYSIS

2. SYSTEM ANALYSIS

System analysis is the process of understanding the current system, identifying problems, and defining the requirements for the proposed system. It helps in designing an efficient and effective Attendance Management System that meets user needs. In this project, system analysis focuses on comparing the **existing manual attendance system** with the **proposed automated system** developed in Python.

2.1 Existing System

- Attendance is recorded manually in registers or sheets.
- Teachers mark each student's presence or absence by hand.
- Calculating total attendance is time-consuming.
- Maintaining records for a large number of students is difficult.
- High chances of human error and misplaced records.

2.2 Proposed System

- Attendance is recorded digitally using Python.
- Users can select academic hierarchy: country, state, district, college, department.
- Attendance can be marked as Present or Absent.
- The system automatically calculates total Present and Absent.
- Student data is managed using classes and objects.
- Records are displayed clearly and can be extended to store in files or databases.

2.3 Feasibility Study

2.3.1 Technical Feasibility

Technical feasibility examines whether the required technology and technical resources are available to develop and run the system. The Attendance Management System can be developed using common technologies such as databases, web or mobile applications, and simple programming languages. Most institutions already have computers and internet access,which makes the system technically possible to implement. Therefore, the technical requirements are manageable and achievable.

2.3.2 Economic Feasibility

Economic feasibility analyzes the cost involved in developing and maintaining the system compared to its benefits. Although there may be initial costs for software development, hardware, or system setup, the long-term benefits such as reduced paperwork, lower administrative effort, and improved efficiency outweigh these costs. Hence, the system is economically feasible.

2.3.3 Operational Feasibility

Operational feasibility checks whether the system will function effectively in the organization and whether users will accept it. Teachers, administrators, or managers can easily use the system after basic training. The system simplifies attendance recording, report generation, and data storage, making daily operations easier and more organized.

SYSTEM SPECIFICATION

3. SYSTEM SPECIFICATION

The **System Specification** describes the hardware, software, and functional requirements needed to develop and run the **Attendance Management System (AMS)** effectively.

3.1 Hardware Requirements

The following hardware components are required to operate the system:

- **Computer/Laptop** with minimum 4 GB RAM and basic processor
- **Hard Disk** with sufficient storage (at least 500 GB)
- **Keyboard and Mouse** for user input
- **Biometric device or RFID scanner** (optional) for automatic attendance marking
- **Printer** (optional) for printing attendance reports
- **Internet connection** for web-based systems

3.2 Software Requirements

The software environment required to develop and run the system includes:

- **Operating System:** Windows, Linux, or macOS
- **Programming Language:** Java, Python, PHP, or similar
- **Database:** MySQL, PostgreSQL, or SQL Server
- **Web Browser:** Chrome, Firefox, or Edge (for web-based systems)
- **Development Tools:** Visual Studio Code, Eclipse, or any IDE

3.3 Software Description

The **Attendance Management System (AMS)** is a software application designed to automate and manage the process of recording and monitoring attendance of students or employees.

1. HTML (HyperText Markup Language)

HTML is used to design the basic structure of the web pages in the Attendance Management System. It creates the layout for forms, login pages, attendance tables, and other interface components. HTML allows users to interact with the system through input fields, buttons, and menus.

2. CSS (Cascading Style Sheets)

CSS is used to style and enhance the appearance of the web pages. It controls elements such as colors, fonts, spacing, and page layout. With CSS, the system interface becomes more attractive, organized, and user-friendly, making it easier for users to navigate the application.

3. Java

Java is used as the backend programming language to handle the main functionality of the system. It processes user requests, manages login authentication, handles attendance calculations, and connects the front-end interface with the database.

4. Python

Python can be used for additional backend processing, automation, or data analysis in the Attendance Management System. It helps in managing data operations, generating reports, and improving system performance with simple and efficient code.

5. SQL (Structured Query Language)

SQL is used to manage the database of the system. It stores important data such as student or employee details, attendance records, login credentials, and reports. SQL allows the system to insert, update, delete, and retrieve data whenever required.

SYSTEM DESIGN

4. SYSTEM DESIGN

System design is the process of planning the structure and components of the Attendance Management System. It defines how the system will work, how data will flow, and how modules will interact. Proper system design ensures the system is efficient, organized, and easy to maintain.

4.1 Input Design

Input design defines how data is entered into the system. The system collects information through web forms created using Flask.

1. Login Input

- **Username** – Text input
- **Password** – Password input

Purpose:
To authenticate users before accessing the system.

2. Add Student Input

This form collects student details.

Fields:

- **Student Name** – Text input
- **Register Number** – Text input
- **Department** – Text input
- **Year** – Number input

Purpose:
To store student information in the database.

3. Add Subject Input

This form allows the admin to add subjects.

Fields:

Subject Name – Text input

Purpose:
To create subjects for attendance tracking.

4. Mark Attendance Input

This form records attendance.

Fields:

- **Student Name** – Dropdown list
- **Subject Name** – Dropdown list
- **Attendance Status** – Select option (Present / Absent)

Purpose:
To record daily attendance of students.

Output design defines how the system presents information to the user.

4.2 Output Design

1. Dashboard Output

The dashboard displays student attendance details including:

- Student Name
- Register Number
- Department
- Year
- Attendance Percentage
- Low Attendance Warning (below 75%)

Purpose:

To monitor student attendance performance.

2. Attendance Report Output

The system can export attendance data as a CSV file.

File name:

`attendance_report.csv`

Contents:

- Student Name
- Register Number
- Department
- Year
- Attendance Percentage

Purpose:

To generate reports for administrative use.

4.3 SYSTEM ARCHITECTURE

The **System Architecture** of the **Attendance Management System (AMS)** describes how different components of the system interact with each other to record, process, and store attendance data. The system generally follows a **three-tier architecture** consisting of the **presentation layer, application layer, and database layer**.

1. Presentation Layer (User Interface)

This is the **front-end layer** where users interact with the system. It includes login pages, attendance forms, dashboards, and report pages. Technologies such as **HTML and CSS** are used to design the interface, allowing administrators, teachers, or staff to easily enter and view attendance information.

2. Application Layer (Business Logic)

This layer handles the **core processing and functionality** of the system. It processes user requests, performs validations, calculates attendance percentages, and manages system operations. Programming languages like **Java or Python** are used in this layer to implement the logic that connects the user interface with the database.

3. Database Layer (Data Storage)

The **database layer** stores all the system data such as user details, attendance records, login credentials, and reports. A database management system using **SQL** is used to insert, update, retrieve, and manage the stored data securely.

SYSTEM IMPLEMENTATION

5.SYSTEM IMPLEMANTATION

The **System Implementation** phase involves developing, testing, and deploying the **Attendance Management System (AMS)** so that it can be used effectively by users such as administrators, teachers, or staff. This phase ensures that the designed system is properly built and functions as expected.

5.1 System Development

In this stage, the system is developed using appropriate programming languages and tools.

HTML is used to design the structure of web pages such as login pages and attendance forms.

CSS is used to style the pages and improve the user interface.

Java or Python is used for backend programming to process user requests and handle system operations.

SQL is used to create and manage the database that stores attendance records and user information.

5.2 Database Implementation

A database is created to store important data such as:

Student or employee details

Attendance records

Login credentials

Attendance reports

Tables are designed using **SQL**, and relationships between tables are established to ensure efficient data management.

5.3 System Deployment

After successful testing, the system is installed and made available for users. It may be deployed on a **local server or web server**, allowing authorized users to access the system through a browser.

5.4 User Training and Maintenance

Users are provided with basic training on how to use the system for marking attendance, viewing reports, and managing records. Regular maintenance is also required to fix bugs, update features, and ensure system security.

SYSTEM TESTING

6. SYSTEM TESTING

System Testing is the process of checking whether the **Attendance Management System (AMS)** works correctly and meets all the required functionalities before it is used by real users. It helps identify errors, bugs, or issues in the system and ensures that the system performs as expected.

6.1 Unit Testing

Unit testing is performed to test **individual modules or components** of the system separately.

Examples:

Testing the login module

Testing the student/employee registration module

Testing the attendance marking module

This ensures that each part of the system functions correctly.

Integration testing checks whether **different modules work properly when combined together**.

Examples:

Checking if attendance data entered in the interface is correctly stored in the database

Ensuring the login module connects properly with the database

This testing ensures smooth communication between system components.

6.2 System Testing

System testing evaluates the **entire system as a whole** to verify that all features work correctly together.

Examples:

Logging into the system

Marking attendance

Viewing attendance records

Generating reports

This ensures that the full system works according to the requirements.

6.3 User Acceptance Testing (UAT)

User Acceptance Testing is performed by **actual users such as teachers or administrators** to verify that the system meets their needs and expectations. Feedback from users helps identify improvements before the system is officially deployed.

CONCLUSION

7. CONCLUSION

The **Attendance Management System (AMS)** is an efficient and reliable solution for managing attendance in educational institutions and organizations. It replaces the traditional manual method of recording attendance with a computerized system, making the process faster, more accurate, and easier to manage.

By using technologies such as **HTML, CSS, Java, Python, and SQL**, the system provides a user-friendly interface, secure data storage, and effective processing of attendance information. It allows administrators and teachers to easily record attendance, maintain records, and generate reports whenever needed.

This project demonstrates practical application of Python concepts such as loops, conditional statements, functions, classes, and objects. It provides a strong foundation for developing more advanced systems with features like database integration, graphical user interface, and automated reporting in the future.

AMS developed in Python successfully automates the process of recording and managing student attendance. By replacing traditional manual registers, the system saves time, reduces errors, and provides accurate attendance records for each student and department.

Overall, the Attendance Management System helps reduce paperwork, minimize human errors, and improve the efficiency of attendance tracking. It provides better data management, saves time, and supports effective monitoring of attendance, making it a valuable tool for modern institutions and organizations.

FUTURE ENHANCEMENT

8. FUTURE ENHANCEMENT

The current **Attendance Management System** developed using Python and the Flask framework works efficiently for managing student attendance. However, several improvements can be implemented in the future to enhance its functionality and usability.

Bulk Student Import

A feature can be added to upload student data using CSV or Excel files, which will make it easier to add multiple students at once.

Student Login Portal

Students can be provided with their own login portal to view their attendance percentage and reports.

Backup and Data Recovery System

A backup system can be implemented to prevent data loss and allow recovery in case of system failure.

These future enhancements will make the system more scalable, secure, and suitable for large educational institutions.

Mobile Application Integration

Developing a mobile app version of the system to allow students and employees to mark attendance remotely or receive real-time notifications.

Biometric and RFID Integration

Adding fingerprint scanners, facial recognition, or RFID cards to automate attendance marking and reduce manual intervention.

Cloud-Based System

Migrating the system to the cloud to allow centralized access from multiple locations, improve data security, and enable easy backup and recovery.

BIBLIOGRAPHY

References

- **W3Schools "HTML, CSS, SQL, and Java Tutorials."** Available at: https://www.w3schools.com
- Oracle Documentation. **"Java Platform, Standard Edition Documentation."** Available at: https://docs.oracle.com/javase/
- Python Software Foundation. **"Python Official Documentation."** Available at: https://docs.python.org
- **MDN Web Docs** "HTML, CSS, and JavaScript Reference." Available at: https://developer.mozilla.org
- **Bootstrap Documentation** "Bootstrap Front-End Framework." Available at: https://getbootstrap.com/docs/

APPENDIX

10.APPENDIX

10.1 Sample Code

```
from flask import Flask, render_template, request, redirect, url_for, flash, session, Response
from flask_sqlalchemy import SQLAlchemy
from werkzeug.security import generate_password_hash, check_password_hash
from datetime import datetime
import csv
from io import StringIO

app = Flask(__name__)
app.config['SECRET_KEY'] = 'supersecretkey'
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///attendance.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db = SQLAlchemy(app)

# ------------------ MODELS ------------------

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(100), unique=True)
    password = db.Column(db.String(200))
    role = db.Column(db.String(20))

class Student(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100))
    reg_no = db.Column(db.String(50), unique=True)
    department = db.Column(db.String(100))
    year = db.Column(db.Integer)
    attendance = db.relationship('Attendance', backref='student', lazy=True)

class Subject(db.Model):
```

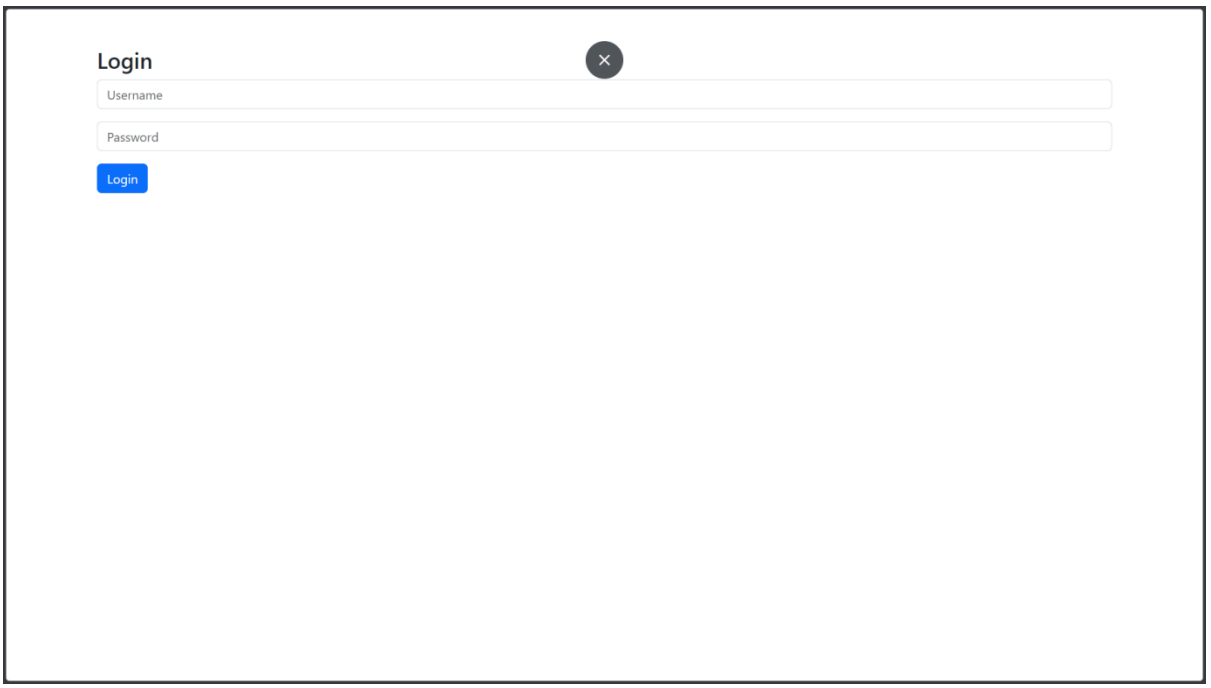
```
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100))
    attendance = db.relationship('Attendance', backref='subject', lazy=True)

class Attendance(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    student_id = db.Column(db.Integer, db.ForeignKey('student.id'))
    subject_id = db.Column(db.Integer, db.ForeignKey('subject.id'))
    status = db.Column(db.String(10))
    date = db.Column(db.Date, default=datetime.utcnow)

# ------------------ AUTH ------------------

@app.route('/', methods=['GET','POST'])
def login():
    if request.method == 'POST':
        user = User.query.filter_by(username=request.form['username']).first()
        if user and check_password_hash(user.password, request.form['password']):
            session['role'] = user.role
            return redirect(url_for('dashboard'))
        flash("Invalid Credentials")
    return render_template("login.html")

@app.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('login'))

# ------------------ DASHBOARD ------------------

@app.route('/dashboard')
def dashboard():
    students = Student.query.all()
    student_data = []
```

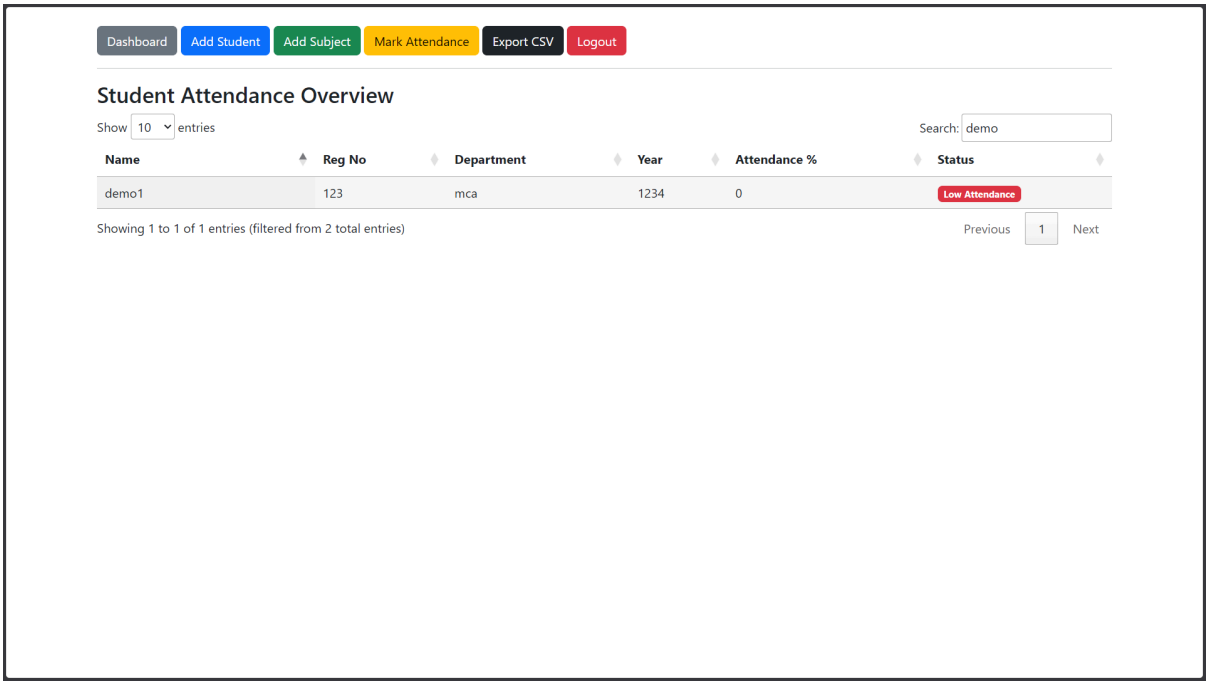
```
    for s in students:
        total = len(s.attendance)
        present = len([a for a in s.attendance if a.status == "Present"])
        percentage = (present/total)*100 if total > 0 else 0
        student_data.append({
            "student": s,
            "percentage": round(percentage,2),
            "low": percentage < 75
        })

    return render_template("dashboard.html", student_data=student_data)

# ------------------ ADD DATA ------------------

@app.route('/add-student', methods=['GET','POST'])
def add_student():
    if request.method == 'POST':
        s = Student(
            name=request.form['name'],
            reg_no=request.form['reg_no'],
            department=request.form['department'],
            year=request.form['year']
        )
        db.session.add(s)
        db.session.commit()
        return redirect(url_for('dashboard'))
    return render_template("add_student.html")

@app.route('/add-subject', methods=['GET','POST'])
def add_subject():
    if request.method == 'POST':
        db.session.add(Subject(name=request.form['name']))
        db.session.commit()
```

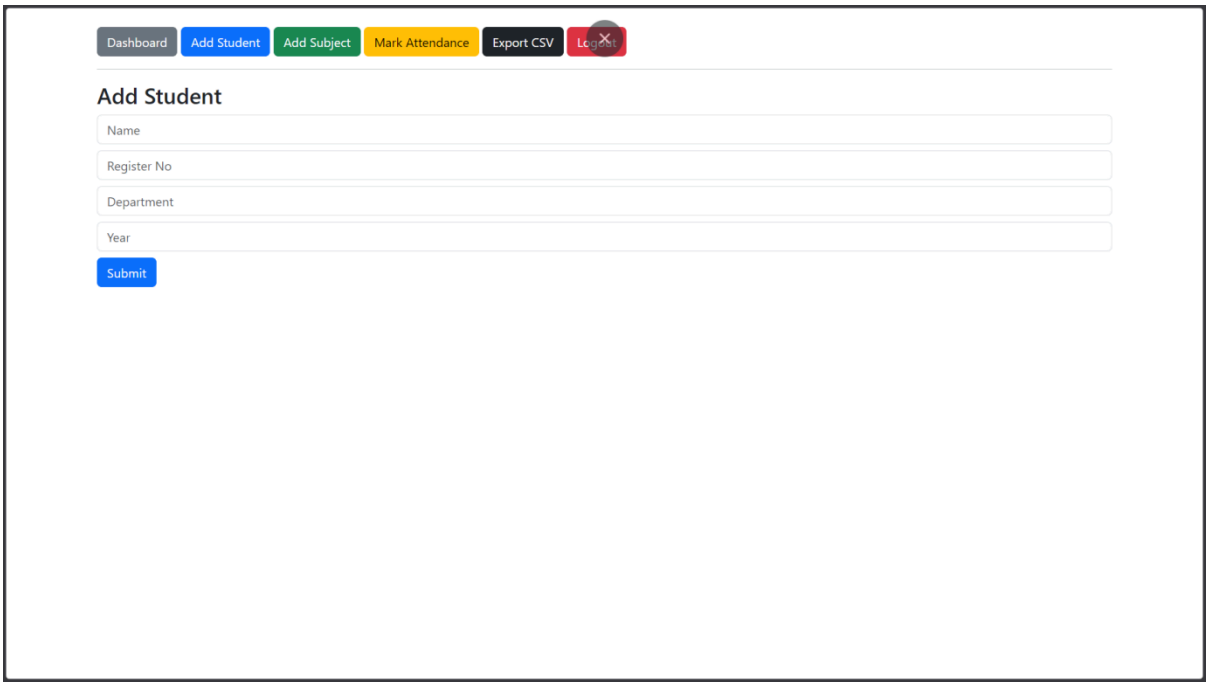
```
        return redirect(url_for('dashboard'))
    return render_template("add_subject.html")

@app.route('/mark-attendance', methods=['GET','POST'])
def mark_attendance():
    students = Student.query.all()
    subjects = Subject.query.all()
    if request.method == 'POST':
        a = Attendance(
            student_id=request.form['student'],
            subject_id=request.form['subject'],
            status=request.form['status']
        )
        db.session.add(a)
        db.session.commit()
        return redirect(url_for('dashboard'))
    return render_template("mark_attendance.html", students=students, subjects=subjects)

# ------------------ EXPORT CSV ------------------

@app.route('/export')
def export():
    si = StringIO()
    cw = csv.writer(si)
    cw.writerow(["Name","Reg No","Department","Year","Percentage"])

    students = Student.query.all()
    for s in students:
        total = len(s.attendance)
        present = len([a for a in s.attendance if a.status == "Present"])
        percentage = (present/total)*100 if total > 0 else 0
        cw.writerow([s.name, s.reg_no, s.department, s.year, round(percentage,2)])

    output = Response(si.getvalue(), mimetype="text/csv")
```

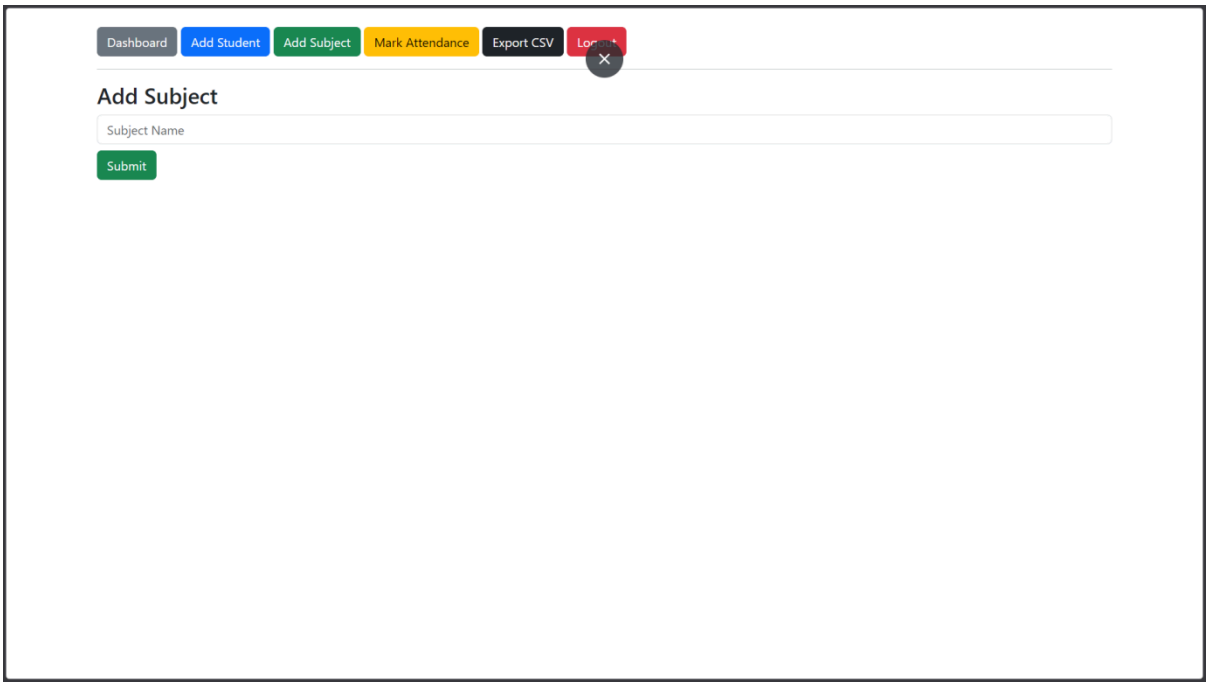
```
    output.headers["Content-Disposition"] = "attachment; filename=attendance_report.csv"
    return output

if __name__ == "__main__":
    with app.app_context():
        db.create_all()
        if not User.query.first():
                                                db.session.add(User(username="admin",
password=generate_password_hash("admin123"), role="admin"))
            db.session.commit()
    app.run(debug=True)
```

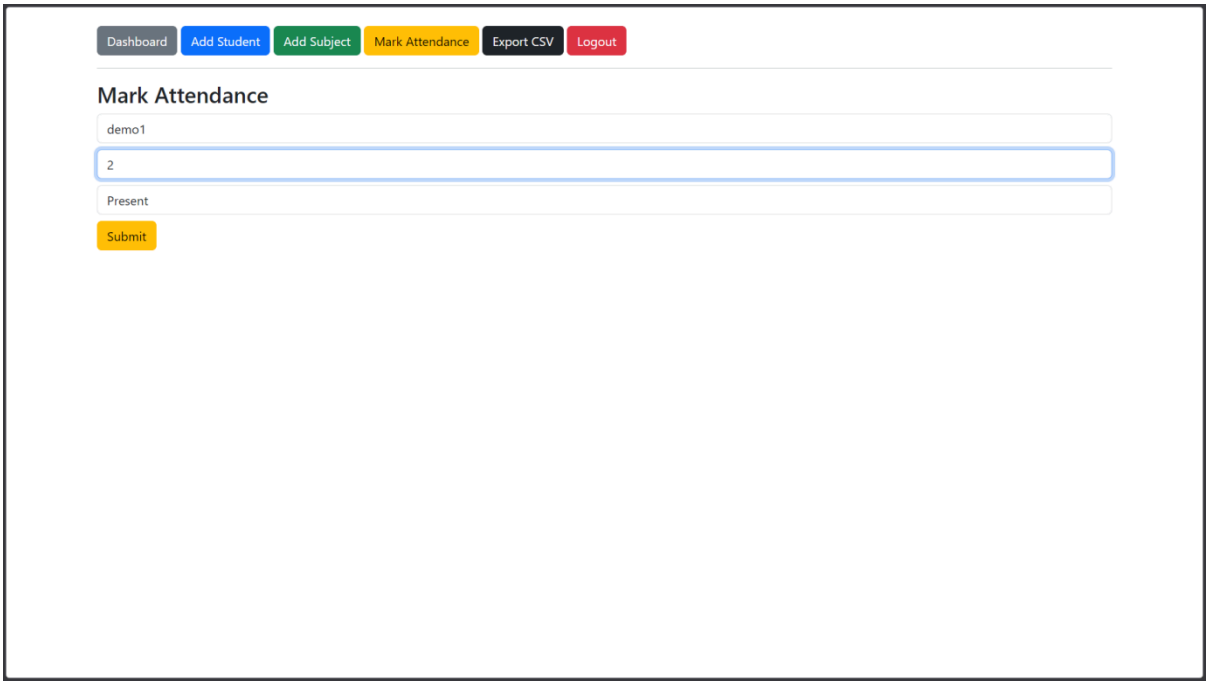
10.2 Sample Output

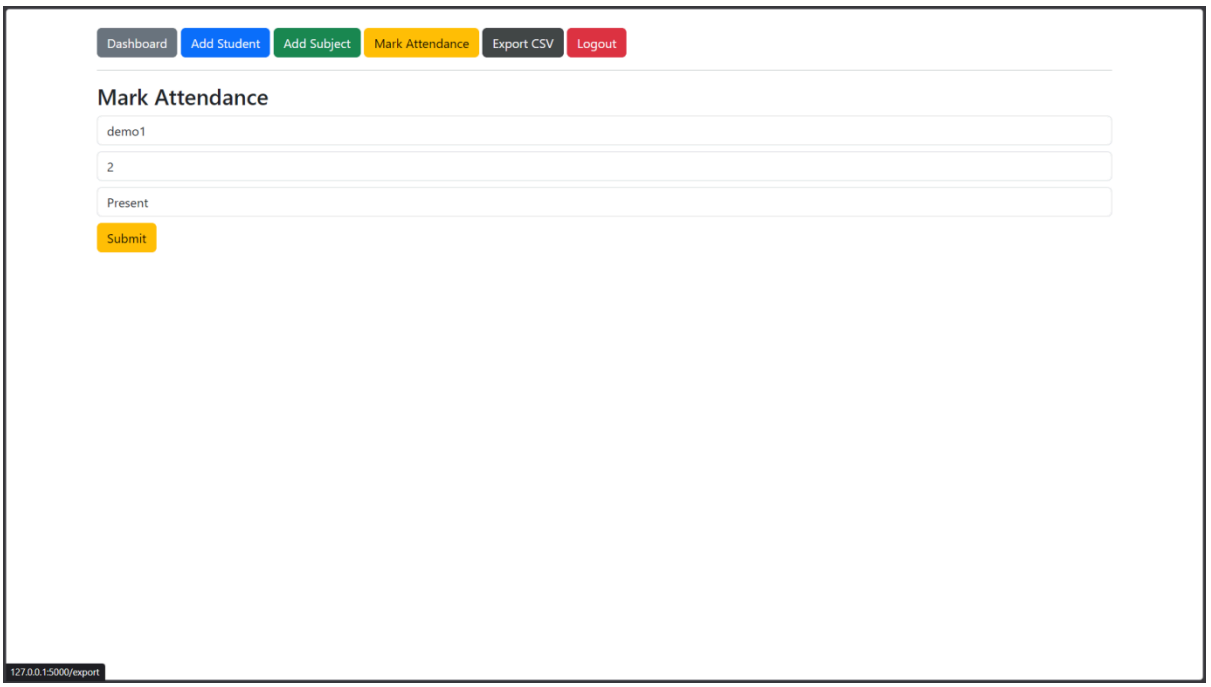





Dashboard
Add Student
Add Subject
Mark Attendance
Export CSV
Add Student
Name
Register No
Department
Year
Submit


Dashboard
Add Student
Add Subject
Mark Attendance
Export CSV
Add Subject
Subject Name
Submit


Dashboard
Add Student
Add Subject
Mark Attendance
Export CSV
Logout
Mark Attendance
demo1
2
Present
Submit


Dashboard
Add Student
Add Subject
Mark Attendance
Export CSV
Logout
Mark Attendance
demo1
2
Present
Submit
127.0.0.1:5000/export